# VLSI BACKEND FILE FORMATS

## A Practical Guide

Akarshan Arora

ai

# CONTENTS


**1 Chapter1: Introduction: ## What's in This Book?hy File Formats Matter in VLSI Backend** **2**
1.1 Why Do You Need to Learn About File Formats? . . . 2
1.2 Who Is This Book For? . . . 3
1.3 Ready to Begin? . . . 3

**2 Chapter 2: Netlists (Verilog and SPICE)** **4**
2.1 What is a Netlist? . . . 4
2.2 Types of Netlists . . . 4
2.3 When and Where Are Netlists Used? . . . 5
2.4 Common File Extensions . . . 5
2.5 Summary . . . 6

**3 Chapter 3: DEF Files (Design Exchange Format)** **7**
3.1 What is a DEF File? . . . 7
3.2 Key Sections of a DEF File . . . 7
3.3 Example DEF Snippet . . . 7
3.4 How to Read a DEF File . . . 8
3.5 Use Cases of DEF Files . . . 10
3.6 Summary . . . 10

**4 Chapter 4: LEF Files (Library Exchange Format)** **11**
4.1 What is a LEF File? . . . 11
4.2 Key Sections in a LEF File . . . 11
4.3 Example LEF Snippets . . . 12
4.4 Understanding LEF Details (Expanded) . . . 13
4.5 LEF and DEF Together . . . 15
4.6 Summary . . . 16

**5 Chapter 5: LIB Files (Timing and Power Libraries)** **17**
5.1 What is a LIB File? . . . 17
5.2 Key Elements of a LIB File . . . 17
5.3 What is PVT? . . . 17
5.4 LIB File Example: Combinational Cell . . . 18
5.5 LIB File Example: Sequential Cell (Flip-Flop) . . . 19
5.6 How to Read 2D Lookup Tables . . . 20
5.7 Summary . . . 21

**6 Chapter 6: SPEF Files (Standard Parasitic Exchange Format)** **22**
6.1 What is a SPEF File? . . . 22
6.2 Why is SPEF Important? . . . 22
6.3 Key Sections in SPEF . . . 23


Akarshan Arora

6.4 Example SPEF File (Corrected and Expanded) . . . 23
6.5 Explanation of Updated Example . . . 24
6.6 Header . . . 24
6.7 Name Mapping . . . 24
6.8 *I Section (Instance Information) . . . 24
6.9 *D_NET Section . . . 24
6.10 Summary . . . 25

**7 Chapter 7: GDSII Files (Graphic Data System II) 26**
7.1 What is a GDSII File? . . . 26
7.2 Why is GDSII Important? . . . 26
7.3 Key Concepts in GDSII . . . 26
7.4 Sample GDSII Elements (Textual Representation) . . . 27
7.5 How to Read GDSII (Using Viewer) . . . 27
7.6 Summary . . . 28

**8 Chapter 8: OASIS Files (Open Artwork System Interchange Standard) 29**
8.1 What is an OASIS File? . . . 29
8.2 Why Use OASIS Instead of GDSII? . . . 29
8.3 Structure of an OASIS File . . . 29
8.4 Sample OASIS Elements (Textual Representation) . . . 30
8.5 Tools Supporting OASIS . . . 30
8.6 When to Use GDSII vs OASIS . . . 30
8.7 Summary . . . 30

**9 Chapter 9: AOCV Files (Advanced On-Chip Variation) 31**
9.1 What is AOCV? . . . 31
9.2 Why is AOCV Important? . . . 31
9.3 AOCV vs. OCV . . . 31
9.4 AOCV Table Format . . . 32
9.5 How to Read AOCV Tables . . . 32
9.6 How AOCV is Used in Tools . . . 32
9.7 Summary . . . 33

**10 Chapter 10: LVF Files (Liberty Variation Format) 34**
10.1 What is LVF? . . . 34
10.2 Why is LVF Important? . . . 34
10.3 LVF vs AOCV vs OCV . . . 34
10.4 Structure of an LVF File . . . 34
10.5 Example Snippet of an LVF File . . . 35
10.6 How to Read an LVF File . . . 35
10.7 How Tools Use LVF . . . 35
10.8 Summary . . . 36

**11 Chapter 11: DSPF Files (Detailed Standard Parasitic Format) 37**
11.1 What is a DSPF File? . . . 37
11.2 Why is DSPF Important? . . . 37
11.3 Structure of a DSPF File . . . 37
11.4 Sample DSPF File . . . 38
11.5 How to Read DSPF . . . 38
11.6 DSPF vs SPEF . . . 39
11.7 Tools That Use DSPF . . . 39
11.8 Summary . . . 39

**12 Chapter 12: VCD Files (Value Change Dump) 40**

Akarshan Arora

12.1 What is a VCD File? . . . 40
12.2 Why is VCD Important? . . . 40
12.3 Structure of a VCD File . . . 40
12.4 Sample VCD File (Complex Example) . . . 41
12.5 How to Read a VCD File . . . 42
12.6 Use in Power Analysis . . . 42
12.7 Tools That Use VCD . . . 42
12.8 Summary . . . 43

**13 Chapter 13: SAIF Files (Switching Activity Interchange Format) 44**
13.1 What is a SAIF File? . . . 44
13.2 Why is SAIF Important? . . . 44
13.3 Structure of a SAIF File . . . 44
13.4 Sample SAIF File (Detailed Example) . . . 45
13.5 How to Read a SAIF File . . . 46
13.6 Comparison with VCD . . . 46
13.7 Summary . . . 47

**14 Chapter 14: FSDB Files (Fast Signal Database) 48**
14.1 What is an FSDB File? . . . 48
14.2 Why is FSDB Important? . . . 48
14.3 Tools that Use FSDB . . . 48
14.4 Generating FSDB Files . . . 48
14.5 How to Read FSDB Files . . . 49
14.6 FSDB and Power Estimation . . . 49
14.7 Summary . . . 49

**15 Chapter 15: Comparing FSDB, VCD, and SAIF Formats 50**
15.1 Overview . . . 50
15.2 Feature Comparison Table . . . 50
15.3 When to Use Each Format . . . 50
15.4 Conversion and Interoperability . . . 51
15.5 Summary . . . 51

Akarshan Arora

Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)

This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License.

You are free to:

- Share — copy and redistribute the material in any medium or format
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.

Under the following terms:

- Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- ShareAlike — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Full License Text: https://creativecommons.org/licenses/by-sa/4.0/legalcode

Author: Akarshan Arora

Akarshan Arora

CHAPTER

ONE

# CHAPTER1: INTRODUCTION: ## WHAT'S IN THIS BOOK?HY FILE FORMATS MATTER IN VLSI BACKEND

When we design modern chips, we go through many complex steps. Some parts involve writing logic (this is called the frontend). Other parts involve turning that logic into actual structures on silicon — this is called the **backend**. The backend is where the design becomes real: where we place standard cells, route wires, and ensure the chip will function reliably under timing, power, and physical constraints.

To handle this complexity, backend tools use many different kinds of **file formats**. These formats act as the **language** between design tools. Each format contains specific types of information, and each one plays a unique role in the chip design flow.

Let's look at a few examples:

- A **DEF** file describes where each component is placed on the chip.
- A **LEF** file defines the shape and pin location of each standard cell.
- A **SPEF** file holds detailed information about parasitic resistance and capacitance.
- A **.lib** file describes how long a gate takes to switch and how much power it uses.

You don't need to memorize these — this book will walk you through each one, step by step.

## 1.1 Why Do You Need to Learn About File Formats?

If you're working in VLSI backend — whether in physical design, signoff, or EDA tool support — you will often run into these files:

- You might be debugging a timing violation and need to look into a **.lib** or **SPEF** file.
- You may need to write a script to extract coordinates from a **DEF** file.
- You may need to compare two **SDC** files to check if constraints are consistent.

Even if you never write these files yourself, understanding them will:

- Help you debug problems faster
- Let you automate repetitive tasks
- Improve your confidence when working with tools and teams

### 1.1.1 What's in This Book?

This book is a **beginner-friendly guide** to the most commonly used file formats in VLSI backend. Each chapter focuses on one format and explains:

- What the file is used for

Akarshan Arora

- Where it fits in the design flow
- What it looks like inside (with real examples)
- Basic tools or scripts to parse or use it

This book doesn't assume you know any coding, but we'll show you small Python snippets that can help automate simple tasks. The goal is not to overwhelm you, but to **get you comfortable** with these essential files.

## 1.2 Who Is This Book For?

This book is written for:

- Fresh graduates or interns in VLSI
- Engineers switching to the backend domain
- Software or CAD engineers supporting VLSI teams
- Anyone who wants to understand the structure of chip data

Whether you're working in a big company or just starting out, this book will give you a strong foundation in the files that power the backend world.

## 1.3 Ready to Begin?

Let's start our journey with the **netlist** — the backbone of every chip design.

Akarshan Arora

CHAPTER

TWO

# CHAPTER 2: NETLISTS (VERILOG AND SPICE)

## 2.1 What is a Netlist?

A **netlist** is a textual representation of an electronic circuit. It describes components such as logic gates, flip-flops, or transistors, and how they are interconnected using electrical nets (wires). In VLSI backend design, netlists provide the logical blueprint that tools use to construct the actual physical layout and perform various analyses.

A netlist **does not include physical location data** — only the logical connectivity. It is generated:

- After **logic synthesis** (from RTL to gate-level netlist)
- After **layout extraction** (from physical design to transistor-level SPICE netlist)

---

## 2.2 Types of Netlists

### 2.2.1 Gate-Level Netlists (Verilog)

These are written in Verilog and include only primitive gates or standard cells. The design hierarchy is usually flattened or partially flattened, and all modules are mapped to library cells.

There are different flavors:

- **Pre-layout netlist (post-synthesis):** Generated after synthesis. Used for pre-layout STA.
- **Post-layout netlist (post-place and route):** Includes additional buffers, hold-fix cells, etc. Used for signoff STA and IR drop.
- **Functional netlist (with behavioral code):** Used in simulation but **not** for physical design.

**Example 1: Basic gate-level netlist (combinational)**

```
module and_or_gate (input A, B, C, output Y);
  wire net1;
  AND2X1 U1 (.A(A), .B(B), .Y(net1));
  OR2X1  U2 (.A(net1), .B(C), .Y(Y));
endmodule
```

This circuit performs: Y = (A & B) | C

**Example 2: Sequential logic (flip-flop)**

```
module dff_example (input CLK, D, output Q);
  DFFPOSX1 U1 (.CLK(CLK), .D(D), .Q(Q));
endmodule
```

Varshan Arora

This describes a positive-edge triggered D flip-flop. Such netlists are key to STA.

### 2.2.2 Transistor-Level Netlists (SPICE)

These are more detailed and show the design at the device level — individual NMOS and PMOS transistors, their sizes, and connections.

**Example SPICE netlist snippet:**

```
M1 out in vdd vdd PMOS L=45n W=120n
M2 out in gnd gnd NMOS L=45n W=60n
```

This describes a CMOS inverter:

- M1 is a PMOS transistor pulling up
- M2 is an NMOS transistor pulling down

SPICE netlists are generated during parasitic extraction and used for:

- Post-layout simulation
- EMIR signoff for analog blocks

---

## 2.3 When and Where Are Netlists Used?

Netlists are central to many backend processes:

| Use Case | Type of Netlist | Purpose |
|---|---|---|
| **Synthesis Output** | Gate-level Verilog | Input to physical design tools |
| **Static Timing Analysis (STA)** | Gate-level Verilog | Analyze timing delays |
| **Power Analysis** | Gate-level Verilog | Estimate switching and leakage power |
| **Equivalence Checking** | RTL vs Gate-level Verilog | Ensure logic equivalence post-synthesis |
| **Post-layout Simulation** | SPICE | Validate signal integrity |
| **IR/EM/Noise Analysis** | SPICE | High-fidelity current modeling |

---

## 2.4 Common File Extensions

| Extension | Description | Example Snippet |
|---|---|---|
| .v | Verilog (can be RTL or gate-level) | AND2X1 U1 (.A(a), .B(b), .Y(y)); |
| .vg | Flattened gate-level Verilog (post-synthesis) | DFFPOSX1 U2 (.CLK(clk), .D(din), .Q(qout)); |
| .sv | SystemVerilog (often used in RTL stage) | logic [3:0] a; always_ff @(posedge clk) a <= b; |
| .sp | SPICE netlist (device-level) | M1 out in vdd vdd PMOS L=45n W=120n |
| .cir | SPICE-compatible format | .SUBCKT INV in out vdd gnd \n M1 out in vdd vdd PMOS |

These extensions help identify what stage or abstraction level the netlist belongs to and what kind of tool should process it.

---

## 2.5 Summary

Understanding netlists is essential for anyone working in VLSI backend. Whether debugging setup violations, analyzing power, or verifying logic, you'll often be reading or modifying Verilog or SPICE netlists. Each type of netlist has its own role, and knowing how to interpret them gives you a solid foundation for working with backend tools.

In the next chapter, we'll move from logic-level to physical layout and explore how component placement is captured in the **DEF (Design Exchange Format)** file.

Akarshan Arora

CHAPTER

THREE

# CHAPTER 3: DEF FILES (DESIGN EXCHANGE FORMAT)

## 3.1 What is a DEF File?

A **DEF (Design Exchange Format)** file describes the **physical layout** of a VLSI design. While netlists define logical connections, DEF files specify where components are placed on the chip and how routing is done. It is widely used during **Place and Route (P&R)** stages.

DEF files are often used together with **LEF (Library Exchange Format)** files, which define cell abstracts—cell dimensions, pin locations, and routing blockages.

---

## 3.2 Key Sections of a DEF File

- VERSION: Specifies the DEF version.
- DIVIDERCHAR, BUSBITCHARS: Define bus syntax.
- DESIGN: Name of the design block or chip.
- DIEAREA: Bounding box coordinates of the chip.
- COMPONENTS: List of standard cells/macros and their placements.
- PINS: Locations, directions, and usages of primary I/O pins.
- NETS: Describes signal nets and their detailed routing.
- SPECIALNETS: Dedicated routing for power and ground nets.
- ROWS, TRACKS, VIAS: Layout infrastructure details.

---

## 3.3 Example DEF Snippet

```
VERSION 5.8 ;
DIVIDERCHAR "/" ;
BUSBITCHARS "[]" ;
DESIGN top ;
DIEAREA ( 0 0 ) ( 4000 3000 ) ;

COMPONENTS 4 ;
 - U1 AND2X1 + PLACED ( 500 1000 ) N ;
```

(continues on next page)

Akarshan Arora

(continued from previous page)

```
  - U2 OR2X1  + PLACED ( 1000 1000 ) N ;
  - U3 DFFPOSX1 + PLACED ( 1500 1200 ) FN ;
  - U4 INVX1 + PLACED ( 2000 1300 ) S ;
END COMPONENTS

PINS 5 ;
  - clk + NET clk + DIRECTION INPUT + USE SIGNAL + PLACED ( 100 100 ) N ;
  - rst + NET rst + DIRECTION INPUT + USE SIGNAL + PLACED ( 200 100 ) N ;
  - out + NET out + DIRECTION OUTPUT + USE SIGNAL + PLACED ( 300 100 ) N ;
  - VDD + NET VDD + DIRECTION INOUT + USE POWER + PLACED ( 0 0 ) N ;
  - VSS + NET VSS + DIRECTION INOUT + USE GROUND + PLACED ( 0 50 ) N ;
END PINS

NETS 4 ;
  - net1 ( U1 Y ) ( U2 A ) + ROUTED metal2 ( 520 1020 ) ( 1000 1020 ) ;
  - net2 ( U2 Y ) ( U3 D ) + ROUTED metal2 ( 1020 1020 ) ( 1500 1220 ) ;
  - net3 ( U3 Q ) ( U4 A ) ( out ) + ROUTED metal2 ( 1520 1220 ) ( 2000 1320 ) ( 300 100 ) ;
  - clk ( clk ) ( U3 CLK ) + ROUTED metal1 ( 100 100 ) ( 1500 1200 ) ;
END NETS

SPECIALNETS 2 ;
  - VDD + USE POWER
    + ROUTED metal1 ( 0 0 ) ( 4000 0 )
    ( U1 VDD ) ( U2 VDD ) ( U3 VDD ) ( U4 VDD ) ( VDD ) ;
  - VSS + USE GROUND
    + ROUTED metal1 ( 0 50 ) ( 4000 50 )
    ( U1 VSS ) ( U2 VSS ) ( U3 VSS ) ( U4 VSS ) ( VSS ) ;
END SPECIALNETS

END DESIGN
```

## 3.4 How to Read a DEF File

### 3.4.1 COMPONENTS

- Each component instance is placed at a given (x, y) coordinate and oriented with a direction (e.g., N, S, FN).
  - Example: - U1 AND2X1 + PLACED ( 500 1000 ) N ;
    - U1: Instance name.
    - AND2X1: Cell type (2-input AND gate).
    - (500 1000): Physical location of the cell.
    - N: Orientation (North = no rotation).

### 3.4.2 NETS

Each net describes how a signal is routed between pins of components.

Example:

```
- net1 ( U1 Y ) ( U2 A ) + ROUTED metal2 ( 520 1020 ) ( 1000 1020 ) ;
```

- net1: Net name.
- ( U1 Y ): Connects the Y output pin of component U1.
- ( U2 A ): Connects the A input pin of component U2.
- + ROUTED metal2: Indicates that the connection is routed on **metal layer 2**.
- ( 520 1020 ) ( 1000 1020 ): A horizontal wire from x=520 to x=1000 at constant y=1020.

**Why is the y-coordinate constant?**

- This represents a **horizontal wire segment**.
- In standard VLSI routing practices:
  - **Even-numbered metal layers** (like metal2) are typically reserved for **horizontal routing**.
  - **Odd-numbered metal layers** (like metal1) are used for **vertical routing**.
- Keeping y constant ensures the wire stays on a single horizontal track, improving routing efficiency and DRC compliance.

**How is the wire width derived?**

- The **DEF file specifies only wire path coordinates**, not width.
- The actual width comes from **LEF files**, where metal layers define:
  - Default width
  - Spacing rules
  - Resistance/capacitance per unit length
- For example, metal2 in LEF might specify a default width of 0.14 microns. When the router uses this layer, it automatically assumes this width unless overridden.

**Pin positions and offsets**

- Although U1 is placed at (500, 1000), the wire starts at (520, 1020). This is due to:
  - **Pin offset** within the standard cell defined in the LEF.
  - The pin Y on U1 may be located 20 units right and 20 units up from the cell origin.
  - These pin offsets enable precise and compact routing.

### 3.4.3 SPECIALNETS

Used for power and ground distribution.

Example:

```
- VDD + USE POWER
  + ROUTED metal1 ( 0 0 ) ( 4000 0 )
  ( U1 VDD ) ( U2 VDD ) ( U3 VDD ) ( U4 VDD ) ( VDD ) ;
```

- This is a power net routed horizontally across the bottom of the chip.
- It connects to all cells' VDD pins and the primary VDD pin.
- Routed using **metal1**, which is vertical by default, but can have horizontal segments as needed.

Akarshan Arora

Similarly, VSS is routed at y=50.

## 3.5 Use Cases of DEF Files

| Use Case | Purpose |
|---|---|
| Physical Verification | Ensure legal placement |
| Routing Analysis | Check for congestion |
| Power Grid Analysis | Verify power rail connectivity |
| Design Rule Checks (DRC) | Validate spacing and layout rules |
| Antenna Checks | Detect charge accumulation issues |
| Floorplanning Visualization | View layout plan |

## 3.6 Summary

DEF files map the **logical design** onto the **physical chip layout**, crucial for completing backend steps like placement, routing, and verification. Understanding how to read DEF files helps debug layout issues, analyze connectivity, and ensure manufacturability.

Next: We'll explore LEF files to understand cell abstracts and layout constraints.

Akarshan Arora

CHAPTER

FOUR

# CHAPTER 4: LEF FILES (LIBRARY EXCHANGE FORMAT)

## 4.1 What is a LEF File?

A **LEF (Library Exchange Format)** file defines the **physical abstract** of standard cells, macros, and routing layers used in a VLSI design. Unlike full layout files like GDSII or OASIS, LEFs contain only the **essential physical attributes** needed by place-and-route tools. This abstraction makes LEFs lightweight and quick to process.

LEF files are typically provided by standard cell vendors and include:

- Cell dimensions and boundaries
- Pin locations and geometries
- Routing blockages
- Routing layer properties
- Via definitions
- Design rules like spacing, pitch, and width

LEF files work in tandem with DEF files. While the DEF describes how cells are placed and nets are routed, the LEF provides the physical definitions required to interpret those placements and routes.

There are **two main types of LEF files**:

1. **Technology LEF (Tech LEF)**: Describes routing layers, vias, design rules, sites, and other process-specific technology information.
2. **Macro LEF (Cell LEF)**: Describes standard cells or macros including their size, pin shapes, pin directions, and blockages.

---

## 4.2 Key Sections in a LEF File

### 4.2.1 Common Sections:

- VERSION: Specifies the LEF file format version.
- BUSBITCHARS, DIVIDERCHAR: Defines syntax for buses and hierarchical net names.

### 4.2.2 In Technology LEF:

- LAYER: Describes routing layers including direction, pitch, width, and spacing.
- VIA: Details the geometry of vias between layers.
- SITE: Defines standard placement grids and symmetry.

Akarshan Arora

### 4.2.3 In Macro LEF:

- MACRO: Describes a standard cell or macro, including its size, pins, and routing blockages.
- PIN: Located inside a MACRO, defines the geometry and direction of each pin.
- OBS: Routing blockages that the router must avoid.

---

## 4.3 Example LEF Snippets

### 4.3.1 Technology LEF Example

```
VERSION 5.7 ;
BUSBITCHARS "[]" ;
DIVIDERCHAR "/" ;

LAYER metal1
  TYPE ROUTING ;
  DIRECTION VERTICAL ;
  PITCH 0.14 ;
  WIDTH 0.14 ;
  SPACING 0.14 ;
END metal1

LAYER metal2
  TYPE ROUTING ;
  DIRECTION HORIZONTAL ;
  PITCH 0.14 ;
  WIDTH 0.14 ;
  SPACING 0.14 ;
END metal2

VIA VIA12
  LAYER metal1 ;
    RECT 0 0 0.14 0.14 ;
  LAYER cut ;
    RECT 0 0 0.1 0.1 ;
  LAYER metal2 ;
    RECT 0 0 0.14 0.14 ;
END VIA12
```

### 4.3.2 Macro LEF Example

```
MACRO AND2X1
  CLASS CORE ;
  ORIGIN 0 0 ;
  SIZE 1.4 BY 1.2 ;
  SYMMETRY X Y ;
  SITE core ;

  PIN A
    DIRECTION INPUT ;
```

(continues on next page)

(continued from previous page)

```
    PORT
      LAYER metal1 ;
        RECT 0.1 0.6 0.2 0.7 ;
    END
  END A

  PIN B
    DIRECTION INPUT ;
    PORT
      LAYER metal1 ;
        RECT 0.1 0.9 0.2 1.0 ;
    END
  END B

  PIN Y
    DIRECTION OUTPUT ;
    PORT
      LAYER metal1 ;
        RECT 1.2 0.6 1.3 0.7 ;
    END
  END Y

  OBS
    LAYER metal1 ;
      RECT 0.3 0.3 1.1 0.9 ;
  END
END AND2X1
```

## 4.4 Understanding LEF Details (Expanded)

To understand LEF files more deeply, let's walk through each important section using examples from both Technology LEF and Macro LEF snippets.

### 4.4.1 Technology LEF (Tech LEF)

**1. LAYER Definition:**

```
LAYER metal1
  TYPE ROUTING ;
  DIRECTION VERTICAL ;
  PITCH 0.14 ;
  WIDTH 0.14 ;
  SPACING 0.14 ;
END metal1
```

- `LAYER metal1`: Declares a routing layer named `metal1`.
- `TYPE ROUTING`: Specifies that this layer is used for routing (not cut, overlap, etc.).
- `DIRECTION VERTICAL`: Preferred routing direction is vertical.
- `PITCH 0.14`: Distance between adjacent wires is 0.14 microns.

- WIDTH 0.14: Default width of a routed wire on this layer.
- SPACING 0.14: Minimum spacing allowed between wires.

This information ensures consistent wire geometry and spacing across the chip.

**2. VIA Definition:**

```
VIA VIA12
  LAYER metal1 ;
    RECT 0 0 0.14 0.14 ;
  LAYER cut ;
    RECT 0 0 0.1 0.1 ;
  LAYER metal2 ;
    RECT 0 0 0.14 0.14 ;
END VIA12
```

- VIA VIA12: A via that connects metal1 to metal2.
- Three layers are defined:
  - metal1: Contact area on metal1
  - cut: Actual via cut area
  - metal2: Contact area on metal2

The rectangles show the via's x/y dimensions on each layer.

**3. SITE Definition (if present):**

```
SITE core
  CLASS CORE ;
  SIZE 1.4 BY 1.2 ;
  SYMMETRY X Y ;
END core
```

- Defines a base unit used in placement.
- SIZE and SYMMETRY help align cells uniformly.

### 4.4.2 Macro LEF (Cell LEF)

**1. MACRO Block:**

```
MACRO AND2X1
  CLASS CORE ;
  ORIGIN 0 0 ;
  SIZE 1.4 BY 1.2 ;
  SYMMETRY X Y ;
  SITE core ;
```

- MACRO AND2X1: Defines a standard cell named AND2X1.
- CLASS CORE: Indicates it's a logic cell (not IO, pad, etc.).
- ORIGIN 0 0: Lower-left corner of the cell layout.
- SIZE: Specifies the width (1.4µm) and height (1.2µm).
- SYMMETRY: Placement symmetry supported.

- SITE core: Tied to the core site definition.

**2. PIN Blocks:**

```
PIN A
  DIRECTION INPUT ;
  PORT
    LAYER metal1 ;
      RECT 0.1 0.6 0.2 0.7 ;
  END
END A
```

- PIN A: Defines input pin A.
- DIRECTION INPUT: Specifies it as an input.
- PORT/LAYER/RECT: Pin shape lies on metal1 layer, rectangle from (0.1, 0.6) to (0.2, 0.7).

**Similarly:**

- PIN B is another input.
- PIN Y is the output, defined on the right edge of the cell with coordinates near (1.2, 0.6) to (1.3, 0.7).

**3. OBS Block:**

```
OBS
  LAYER metal1 ;
    RECT 0.3 0.3 1.1 0.9 ;
END
```

- OBS (Obstruction): Specifies a region on metal1 that should not be used for routing.
- This helps avoid congestion or design rule violations.

---

## 4.5 LEF and DEF Together

To fully interpret a routed net in DEF, you need the LEF:

From DEF:

```
- net1 ( U1 Y ) ( U2 A ) + ROUTED metal2 ( 520 1020 ) ( 1000 1020 ) ;
```

From LEF:

- The Y pin on U1 may be defined at (1.2, 0.6) to (1.3, 0.7).
- If U1 is placed at (500, 1000), pin Y's absolute position becomes approximately (520, 1020).
- Wire is horizontal on metal2, with default width defined in LEF (0.14).
- Y-coordinates (1020) are constant as the wire runs horizontally — only X changes.
- Width comes from the WIDTH field of metal2 in Tech LEF (here 0.14).

---

Akarshan Arora

## 4.6 Summary

LEF files provide an abstract yet accurate description of standard cells and routing resources, allowing efficient and accurate placement and routing. Understanding LEFs is essential for interpreting DEF files and verifying physical layout constraints.

In the next chapter, we will look at **LIB files**, which describe timing, power, and functional characteristics of standard cells.

Akarshan Arora

CHAPTER

FIVE

# CHAPTER 5: LIB FILES (TIMING AND POWER LIBRARIES)

## 5.1 What is a LIB File?

A **LIB file**, also known as a Liberty file (.lib), is a text-based format used to describe the **timing, power, area, and logical function** of standard cells and macros. It is essential for:

- **Static Timing Analysis (STA)**
- **Synthesis (logic to gates)**
- **Power analysis**
- **Equivalence checking**

LIB files are generated through detailed simulation and characterization by the foundry or library provider. These files are used in digital implementation to understand how fast or power-hungry a cell is.

---

## 5.2 Key Elements of a LIB File

Each LIB file contains the following components:

- **Library**: The top-level container (represents a specific PVT corner like typical, worst, best)
- **Cell**: Represents a single logic cell (like NAND2, DFF)
- **Pin**: Defines characteristics of each input/output pin
- **Function**: Defines logic behavior
- **Timing**: Describes delays from inputs to outputs
- **Power**: Describes power consumption: both leakage and dynamic

---

## 5.3 What is PVT?

**PVT** stands for **Process, Voltage, and Temperature**. Each LIB file is characterized for a specific PVT corner:

- **Process**: Manufacturing variation (typical, slow, fast)
- **Voltage**: Operating voltage (e.g., 1.0V, 0.9V)
- **Temperature**: Ambient condition (e.g., 25°C, 125°C)

Tools use different PVT libraries to perform worst-case and best-case analysis.

---

Akarshan Arora

## 5.4 LIB File Example: Combinational Cell

```
library (typical) {

  cell (AND2X1) {
    area : 1.68;

    pin (A) {
      direction : input;
      capacitance : 0.002;
      internal_power () {
        rise_power ("typical") {
          index_1 ("0.01, 0.05, 0.1");
          index_2 ("0.01, 0.1, 0.2");
          values ( \
            "0.001, 0.0011, 0.0012",
            "0.0012, 0.0013, 0.0014",
            "0.0015, 0.0016, 0.0017" );
        }
        fall_power ("typical") {
          index_1 ("0.01, 0.05, 0.1");
          index_2 ("0.01, 0.1, 0.2");
          values ( \
            "0.0012, 0.0013, 0.0014",
            "0.0014, 0.0015, 0.0016",
            "0.0016, 0.0017, 0.0018" );
        }
      }
    }

    pin (B) {
      direction : input;
      capacitance : 0.002;
    }

    pin (Y) {
      direction : output;
      function : "A & B";
      max_capacitance : 0.02;

      internal_power () {
        rise_power ("typical") { values("0.002") ; }
        fall_power ("typical") { values("0.0021") ; }
      }

      timing () {
        related_pin : "A";
        cell_rise ("typical") {
          index_1 ("0.01, 0.05, 0.1");
          index_2 ("0.01, 0.1, 0.2");
          values ( \
            "0.05, 0.07, 0.09",
            "0.06, 0.08, 0.1",
```

(continues on next page)

(continued from previous page)

```
          "0.07, 0.09, 0.11" );
        }
        cell_fall ("typical") {
          index_1 ("0.01, 0.05, 0.1");
          index_2 ("0.01, 0.1, 0.2");
          values ( \
            "0.06, 0.08, 0.1",
            "0.07, 0.09, 0.11",
            "0.08, 0.1, 0.12" );
        }
      }
    }
  }
}
```

## 5.5 LIB File Example: Sequential Cell (Flip-Flop)

```
cell (DFFPOSX1) {
  area : 2.5;

  ff (IQ, IQN) {
    clocked_on : "CLK";
    next_state : "D";
  }

  pin (CLK) {
    direction : input;
    capacitance : 0.003;
    timing_type : rising_edge;
  }

  pin (D) {
    direction : input;
    capacitance : 0.0025;
  }

  pin (Q) {
    direction : output;
    function : "IQ";
    internal_power () {
      rise_power ("typical") {
        index_1 ("0.01, 0.05, 0.1");
        index_2 ("0.01, 0.1, 0.2");
        values ( \
          "0.002, 0.0022, 0.0024",
          "0.0023, 0.0025, 0.0027",
          "0.0026, 0.0028, 0.0030" );
      }
      fall_power ("typical") {
```

(continues on next page)

(continued from previous page)

```
      index_1 ("0.01, 0.05, 0.1");
      index_2 ("0.01, 0.1, 0.2");
      values ( \
        "0.0021, 0.0023, 0.0025",
        "0.0024, 0.0026, 0.0028",
        "0.0027, 0.0029, 0.0031" );
    }
  }

  timing () {
    related_pin : "CLK";
    timing_sense : positive_unate;
    cell_rise ("typical") {
      index_1 ("0.01, 0.05, 0.1");
      index_2 ("0.01, 0.1, 0.2");
      values ("0.08, 0.1, 0.12") ;
    }
    cell_fall ("typical") {
      index_1 ("0.01, 0.05, 0.1");
      index_2 ("0.01, 0.1, 0.2");
      values ("0.09, 0.11, 0.13") ;
    }
  }
 }
}
```

## 5.6 How to Read 2D Lookup Tables

A 2D table in LIB file represents how delay or power changes with varying:

- **Input transition time** (how fast the input switches)
- **Output load** (capacitance load at output)

### 5.6.1 Syntax Explanation:

```
index_1 ("0.01, 0.05, 0.1");  // input transition times
index_2 ("0.01, 0.1, 0.2");   // output load values
values (
  "0.05, 0.07, 0.09",
  "0.06, 0.08, 0.1",
  "0.07, 0.09, 0.11"
);
```

This table means:

- **Row 1** → transition 0.01 ns: delays are 0.05, 0.07, 0.09 for load 0.01, 0.1, 0.2 respectively
- **Row 2** → transition 0.05 ns: delays are 0.06, 0.08, 0.1
- **Row 3** → transition 0.1 ns: delays are 0.07, 0.09, 0.11

### 5.6.2 More Examples:

- **For index_1 = 0.01**, index_2 = 0.2 → value = 0.09
- **For index_1 = 0.05**, index_2 = 0.1 → value = 0.08
- **For index_1 = 0.1**, index_2 = 0.01 → value = 0.07

This helps us quickly estimate the delay or power for any given input slew and output load.

---

## 5.7 Summary

- LIB files are critical for accurate modeling of delay, power, and functionality.
- Every pin is analyzed for its capacitance, power, direction, and timing behavior.
- Power values vary by slew and load.
- Combinational and sequential cells are both defined with logic functions and delay models.
- These files are consumed by timing, synthesis, and power tools.

In the next chapter, we will explore **SPEF files**, which model parasitic resistance and capacitance of routed nets.

Akarshan Arora

CHAPTER

SIX

# CHAPTER 6: SPEF FILES (STANDARD PARASITIC EXCHANGE FORMAT)

## 6.1 What is a SPEF File?

A **SPEF (Standard Parasitic Exchange Format)** file is used to describe the **parasitic resistances and capacitances** of nets in an integrated circuit after place and route. These parasitics arise from the actual physical wires and vias used to interconnect cells in the layout.

SPEF files are generated by extraction tools (like StarRC, Calibre xACT, etc.) after the layout is completed, and are consumed by STA and power analysis tools.

---

## 6.2 Why is SPEF Important?

When a circuit is designed at RTL level, delays are considered ideal. But real-world wires have:

- **Resistance (R)** – slows down signals
- **Capacitance (C)** – loads the output driver

These effects change the delay of the circuit significantly and impact power too. SPEF provides this real post-layout data.

Without SPEF:

- Timing analysis will be too optimistic or pessimistic.
- Signal integrity (like noise, glitches) cannot be checked.
- Power estimation will miss dynamic IR drop caused by switching wires.

With SPEF, tools can calculate:

- Accurate **point-to-point delays**
- **Voltage drops** (IR)
- **Crosstalk noise**

---

Akarshan Arora

## 6.3 Key Sections in SPEF

1. **Header Information**: Metadata like design name, date, units.
2. **Name Mapping**: Short forms for long instance/net names.
3. **Instance Information**: Pin positions and directions.
4. **Parasitic Data**: RC information for each net (capacitance, resistance).

---

## 6.4 Example SPEF File (Corrected and Expanded)

```
*SPEF "IEEE 1481-1998"
*DESIGN "top"
*DATE "Thu Apr 10 12:00:00 2025"
*VENDOR "ToolVendor"
*PROGRAM "RCXTool"
*VERSION "1.0"
*DIVIDER /
*DELIMITER :
*T_UNIT 1e-12
*C_UNIT 1e-15
*R_UNIT 1

*NAME_MAP
*1 I0/U1
*2 I0/U2
*3 I0/U3
*4 I0/U4
*5 net1
*6 net2
*7 A
*8 B
*9 Y

*I
*1 A 10.0 10.0 INPUT
*2 A 20.0 10.0 INPUT
*3 A 30.0 10.0 INPUT
*4 Y 40.0 10.0 OUTPUT

*D_NET 5 0.15
*C *5 *1:7 0.03
*C *5 *2:7 0.03
*C *5 *3:7 0.03
*C *5 *4:7 0.03
*C *5 0.03
*R *5 *1:7 *2:7 50.0
*R *5 *2:7 *3:7 40.0
*R *5 *3:7 *4:7 30.0
*R *5 *4:7 *5 20.0
```

(continues on next page)

(continued from previous page)

```
*D_NET 6 0.08
*C *6 *4:9 0.04
*C *6 *6 0.04
*R *6 *4:9 *6 10.0
```

## 6.5 Explanation of Updated Example

## 6.6 Header

Standard metadata about units, tool used, and design name.

## 6.7 Name Mapping

Maps short numbers to instances and net names for compactness:

- *1 → U1
- *4:7 → Pin A of U4
- *5 → net1
- *6 → net2

## 6.8 *I Section (Instance Information)

Specifies pins for each instance:

- *1 A 10.0 10.0 INPUT → Instance U1, pin A at (10,10), INPUT
- *4 Y 40.0 10.0 OUTPUT → Instance U4, pin Y at (40,10), OUTPUT

This is useful to trace parasitics back to their physical locations.

## 6.9 *D_NET Section

Each net's parasitic data:

- Capacitances *C at pins (U1:A, U2:A, etc.) and wire
- Resistances *R across segments

For net1, it connects 4 instances (U1 → U2 → U3 → U4):

- Capacitances at each A pin and one wire
- Resistive path formed by 4 resistors, one between each segment

For net2, connects the output pin Y of U4 to an external sink (unnamed node)

Akarshan Arora

## 6.10 Summary

- SPEF provides post-layout parasitic resistance and capacitance.
- Essential for STA and power analysis.
- Realistic delay and power models rely on this data.
- Includes instance-to-pin map, coordinates, and net-level RC ladder.

Next: We'll learn about **GDSII files**, used to view layout.

Akarshan Arora

CHAPTER

SEVEN

# CHAPTER 7: GDSII FILES (GRAPHIC DATA SYSTEM II)

## 7.1 What is a GDSII File?

The **GDSII (Graphic Data System II)** file format is a binary representation used to describe the **physical layout** of an integrated circuit (IC). It is considered the **golden format** for transferring IC layout data from design tools to the foundry for fabrication.

Developed by Calma in the 1970s, GDSII remains widely used in industry despite the introduction of newer formats like OASIS. It stores geometric shapes, layer information, and hierarchy in a compact and efficient manner.

---

## 7.2 Why is GDSII Important?

The GDSII format plays a **critical role in the tapeout process** (the final step before manufacturing). Here's why it's indispensable:

- **Final Handoff to Foundry**: It is the definitive layout file submitted to the fabrication facility. Without GDSII, no silicon chip can be manufactured.
- **Manufacturing Masks Generation**: Foundries generate photolithographic masks directly from GDSII data.
- **Layout vs Schematic (LVS) and Design Rule Check (DRC)**: Physical verification tools like Calibre or PVS use GDSII to ensure the layout is correct and matches the netlist.
- **Visualization and Debug**: Designers use GDS viewers to verify routing, metal fills, standard cell placement, etc.
- **Tool Interoperability**: Almost every physical design tool can generate or read GDSII, making it a universal format in the backend flow.

Because of its hierarchical structure, large designs can be stored efficiently, and changes can be propagated easily.

---

## 7.3 Key Concepts in GDSII

- **Library/Structure**: A design is made up of structures (also known as cells or blocks), such as INVX1, NAND2X2, or TOP.
- **Elements**:
  - **BOUNDARY**: Polygons or rectangles that represent layout shapes (e.g., diffusion, metal).
  - **PATH**: Represents wires or routing tracks.

Akarshan Arora

  - **TEXT**: Labels for identification or debugging.
- **Layers**: Each shape is associated with a layer (like METAL1, POLY, DIFFUSION), using numeric codes defined by the foundry.
- **Coordinates**: Each polygon or path uses X-Y coordinates to define its shape in the layout. These are stored in database units (DBUs), usually in microns.
- **Hierarchy**: Structures can reference other structures (cells within cells), supporting a modular and compact design representation.

---

## 7.4 Sample GDSII Elements (Textual Representation)

Though GDSII is binary, here is a simplified text representation:

```
STRUCTURE INVX1
  BOUNDARY LAYER=1 XY= (0,0)(0,2)(2,2)(2,0)          // Represents diffusion area
  BOUNDARY LAYER=2 XY= (0.5,0.5)(0.5,1.5)(1.5,1.5)(1.5,0.5) // Represents polysilicon gate
  PATH LAYER=5 WIDTH=0.1 XY=(0,1)(2,1)               // Represents metal wire
  TEXT "INVX1" LAYER=65 XY=(1,1)                     // Cell label for readability
ENDSTRUCTURE

STRUCTURE TOP
  SREF INVX1 XY=(10,10)                              // Instantiation of INVX1
  SREF INVX1 XY=(20,10)                              // Another instance of INVX1
ENDSTRUCTURE
```

---

## 7.5 How to Read GDSII (Using Viewer)

GDSII viewers or layout tools help visualize the binary GDSII file:

- **Layer Navigation**: You can isolate specific layers like METAL1, METAL2, etc., to examine routing or via connections.
- **Cell Hierarchy Exploration**: View the hierarchy tree to understand how different blocks are nested.
- **Shape Details**: Clicking a shape reveals its layer, dimensions, coordinates, and net association.
- **Routing and Fill Check**: Easily trace routed nets, check for open nets, and verify metal density compliance.
- **Measurement Tools**: Use rulers and area calculators for debugging and validation.

Common tools:

- **KLayout**: Free and open-source, widely used for inspection and debugging.
- **Calibre DESIGNrev**: From Siemens EDA, supports DRC/LVS integration.
- **Virtuoso Layout Editor**: Cadence tool for interactive editing.
- **Synopsys ICC2 Viewers**: Used in digital P&R flows.

---

Akarshan Gupta

## 7.6 Summary

- **GDSII** is the de facto standard for layout representation in VLSI physical design.
- It stores **geometrical, topological, and hierarchical data** efficiently.
- The format is essential for **tapeout**, physical verification, and fabrication.
- It supports **modular design**, enabling reuse of standard cells and macros.
- Every successful silicon chip begins with a properly generated and verified GDSII file.

Next: We'll explore another important physical file format — **OASIS (Open Artwork System Interchange Standard)** — which is a modern and more compact alternative to GDSII.

Akarshan Arora

CHAPTER

EIGHT

# CHAPTER 8: OASIS FILES (OPEN ARTWORK SYSTEM INTERCHANGE STANDARD)

## 8.1 What is an OASIS File?

**OASIS (Open Artwork System Interchange Standard)** is a modern file format developed to overcome the limitations of GDSII. It provides a **more compact**, **efficient**, and **extensible** representation of integrated circuit layouts, especially important for **large and complex designs** in advanced technology nodes.

Developed by **SEMI (Semiconductor Equipment and Materials International)**, OASIS offers features such as variable-length encoding, support for repeating patterns, and data compression that significantly reduce file size and improve processing speed.

---

## 8.2 Why Use OASIS Instead of GDSII?

While GDSII has been the industry standard for decades, the increasing complexity of chips and the need for efficient data handling have made OASIS more appealing. Here are some reasons:

- **Reduced File Size**: OASIS files are often 5–50 times smaller than their GDSII counterparts.
- **Faster File Transfer**: Smaller size leads to quicker data transfer and loading times.
- **Better Support for Repetition**: Complex layouts often reuse patterns. OASIS allows defining these once and referencing them multiple times.
- **Efficient Data Representation**: Uses compact binary encoding and can store various data types more efficiently.
- **More Layers and Datatypes**: Supports more than 64 layers, which GDSII is typically limited to.

---

## 8.3 Structure of an OASIS File

Like GDSII, OASIS also uses a **hierarchical** structure to describe layout data. The key elements include:

- **Cell Definitions**: Equivalent to structures in GDSII, cells are defined and instantiated.
- **Shapes**: Rectangles, polygons, and paths represent physical layout geometries.
- **Layers and Datatypes**: Each shape is placed on a specific layer and may have an associated datatype.
- **X,Y Coordinates**: Used for placing geometries and instances.
- **Repetition Constructs**: Allow defining a pattern once and using it repeatedly with transformations.
- **Properties**: Can store additional metadata like layer names, net associations, or custom information.

Akanshan Arora

## 8.4 Sample OASIS Elements (Textual Representation)

While OASIS is a binary format, here's a conceptual example similar to how it might be represented in a viewer or decoder:

```
BEGIN CELL inverter
  RECTANGLE LAYER=1 DATATYPE=0 (0,0) (2,2)          // Diffusion
  RECTANGLE LAYER=2 DATATYPE=0 (0.5,0.5) (1.5,1.5)   // Polysilicon
  PATH LAYER=5 WIDTH=0.1 (0,1) (2,1)                // Metal wire
  TEXT "INV" LAYER=65 (1,1)
END CELL

BEGIN CELL top
  PLACEMENT inverter AT (10,10)
  PLACEMENT inverter AT (20,10)
END CELL
```

## 8.5 Tools Supporting OASIS

Most modern physical design and verification tools support OASIS natively:

- **KLayout**: Open-source layout viewer that supports OASIS and GDSII.
- **Cadence Virtuoso**: Full OASIS support for custom design.
- **Calibre**: Siemens EDA tool for DRC/LVS with OASIS support.
- **Synopsys IC Compiler II**: Handles OASIS for physical implementation.

## 8.6 When to Use GDSII vs OASIS

| Criteria | GDSII | OASIS |
|---|---|---|
| File Size | Larger | Smaller |
| Read/Write Speed | Slower | Faster |
| Tool Compatibility | Widely supported | Growing support |
| Hierarchical Support | Good | Excellent |
| Advanced Nodes | Limited efficiency | Highly efficient |

## 8.7 Summary

- **OASIS** is a next-generation layout file format that offers significant efficiency improvements over GDSII.
- It supports **compact file size**, **repetition**, and **metadata**, making it ideal for advanced designs.
- As designs get larger and more complex, OASIS adoption is increasing across the VLSI industry.

Next: We'll move from physical layout files to **AOCV (Advanced On Chip Variation)** format used for timing.

Akarshan A

CHAPTER

NINE

# CHAPTER 9: AOCV FILES (ADVANCED ON-CHIP VARIATION)

## 9.1 What is AOCV?

**AOCV (Advanced On-Chip Variation)** is a method used in **timing analysis** to model the impact of process variations on the delay of standard cells and interconnects. Unlike traditional corner-based analysis or simple OCV (On-Chip Variation), AOCV provides a more fine-grained, location-aware, and path-sensitive method to account for these variations.

AOCV uses **lookup tables** based on path depth and transition time (slew), providing a more accurate representation of timing variations due to manufacturing inconsistencies and environmental conditions.

---

## 9.2 Why is AOCV Important?

Modern technology nodes (especially below 28nm) are significantly affected by process variation. AOCV helps by:

- Providing **realistic timing margins**.
- Avoiding **over-pessimism** seen in traditional OCV.
- Enabling **path-specific** variation modeling.
- Helping meet **timing closure** with better optimization.
- Ensuring **yield-aware design**, by accounting for statistical delay behavior.

---

## 9.3 AOCV vs. OCV

| Feature | OCV | AOCV |
|---|---|---|
| Variation Model | Single margin | Lookup table per cell/arc |
| Parameters | Global margin | Depth, Slew |
| Accuracy | Conservative | More accurate |
| Impact | Over-pessimism | Reduced pessimism |

---

Akarshan Arora

## 9.4 AOCV Table Format

A typical AOCV file is a simple ASCII format used by tools like Primetime. It includes **cell types**, **arc types**, **depth**, **slew**, and **derating factors**.

Example snippet:

```
cell NAND2X1
  from_pin A to_pin Y
    depth 1
      slew 10 derate 0.85
      slew 20 derate 0.88
    depth 2
      slew 10 derate 0.90
      slew 20 derate 0.93

cell DFFPOSX1
  from_pin CLK to_pin Q
    depth 1
      slew 10 derate 0.92
```

Each derate factor tells the tool how to adjust the nominal delay based on path depth and input slew.

---

## 9.5 How to Read AOCV Tables

1. **Cell and Arc**:
   - `cell NAND2X1`: Specifies the library cell.
   - `from_pin A to_pin Y`: Defines the timing arc.
2. **Depth**:
   - `depth 1`: Refers to how deep in the logic path this arc is. A deeper path sees more cumulative variation.
3. **Slew**:
   - Input transition time (10ps, 20ps, etc.)
4. **Derate**:
   - Delay = Nominal × Derate
   - A derate of 0.85 means the delay is 85% of nominal (indicating faster than nominal performance due to variation modeling).

---

## 9.6 How AOCV is Used in Tools

Timing tools like **Primetime** or **Tempus** use AOCV tables during static timing analysis (STA). The derate values are applied during propagation of delays across the design netlist.

```
set_aocvm_model -library my_lib -type max -file my_max.aocvm
set_aocvm_model -library my_lib -type min -file my_min.aocvm
```

Akarshan A

These commands tell the tool to use AOCV for setup and hold analysis respectively.

---

## 9.7 Summary

- **AOCV** helps model process variations more accurately than traditional OCV.
- It uses lookup tables indexed by **depth** and **slew**.
- It reduces pessimism and improves timing closure.
- Tools use AOCV data during STA to derate delays based on context.

Next: We'll explore **Liberty Variation Format (LVF)** for even more advanced statistical timing analysis.

Akarshan Arora

CHAPTER

TEN

# CHAPTER 10: LVF FILES (LIBERTY VARIATION FORMAT)

## 10.1 What is LVF?

**LVF (Liberty Variation Format)** is an extension of the standard Liberty (.lib) file format used in VLSI design. It introduces **statistical variation** into cell timing characterization. This allows for more accurate **statistical static timing analysis (SSTA)** by modeling not just a single delay value, but a **distribution** of delays due to process, voltage, and temperature (PVT) variations.

LVF is critical in modern nodes (FinFET and below), where variability has a significant impact on delay and setup/hold margins.

---

## 10.2 Why is LVF Important?

- **Improved Accuracy**: It captures delay variability more accurately than AOCV.
- **Statistical Timing**: Allows analysis tools to understand the mean and standard deviation of delays.
- **Yield Optimization**: Supports better timing closure targeting high yield.
- **PVT Aware**: Offers delay characterization over PVT corners with statistical modeling.

---

## 10.3 LVF vs AOCV vs OCV

| Feature | OCV | AOCV | LVF |
|---|---|---|---|
| Model | Constant | Depth/Slew Table | Statistical Distribution |
| Accuracy | Low | Medium | High |
| Format | Fixed margin | Derate table | PDF/Statistical tables |
| Used For | Early design | Signoff | Advanced signoff & yield |

---

## 10.4 Structure of an LVF File

LVF files embed extra statistical information within Liberty cell timing arcs. Key keywords include:

- `lu_table_template`: Defines table structure (e.g., slew × load)
- `statistical_variable`: Defines mean and sigma (standard deviation)

Akarshan Arora

- cell, pin, timing: Contain the actual variation-aware timing data

---

## 10.5 Example Snippet of an LVF File

```
lu_table_template(slew_load_lvf) {
  variable_1 : input_transition_time;
  variable_2 : total_output_net_capacitance;
  index_1("0.05, 0.10, 0.15");
  index_2("0.02, 0.05, 0.10");
}

cell(NAND2_X1) {
  pin(Y) {
    timing() {
      related_pin : "A";
      timing_sense : negative_unate;
      cell_rise_stat_mean(slew_load_lvf) {
        values("0.05, 0.10, 0.15",
               "0.06, 0.12, 0.18",
               "0.07, 0.14, 0.21");
      }
      cell_rise_stat_sigma(slew_load_lvf) {
        values("0.005, 0.008, 0.010",
               "0.006, 0.009, 0.011",
               "0.007, 0.010, 0.012");
      }
    }
  }
}
```

This example provides statistical rise delay (mean and sigma) based on input transition (slew) and output load (capacitance).

---

## 10.6 How to Read an LVF File

- cell_rise_stat_mean: Gives **average delay** for each slew-load pair
- cell_rise_stat_sigma: Gives **standard deviation** — captures the spread of delay variation
- **Index values** are lookup points for slew and load
- **Tools interpolate** between values to derive delay distribution

---

## 10.7 How Tools Use LVF

Timing engines like **PrimeTime VX**, **Tempus**, or **GoldTime** can use LVF for:

- Statistical STA (SSTA) where delay is modeled as mean $\pm$ N×sigma

- Timing yield estimation across PVT corners
- Crosstalk-aware timing analysis
- Monte Carlo simulations for worst-case path estimation

---

## 10.8 Summary

- **LVF** brings statistical variation awareness to timing models
- It is more precise than OCV and AOCV
- Uses tables of **mean and sigma** values for delay and transition
- Tools leverage LVF for better signoff and yield analysis

Next: We'll explore **parasitic extraction formats like DSPF and SPEF**, critical for accurate RC modeling and delay computation.

Akarshan Arora

CHAPTER

ELEVEN

# CHAPTER 11: DSPF FILES (DETAILED STANDARD PARASITIC FORMAT)

## 11.1 What is a DSPF File?

**DSPF (Detailed Standard Parasitic Format)** is a parasitic extraction file format that provides **detailed RC values** (resistance and capacitance) of interconnects in a design. It is commonly generated after layout and used for **accurate post-layout timing analysis**, **power estimation**, and **signal integrity analysis**.

It is more granular than SPEF, often capturing transistor-level parasitics and is especially useful for custom and analog designs.

---

## 11.2 Why is DSPF Important?

- **Post-Layout Timing**: Includes realistic parasitics, essential for accurate delays.
- **Power Estimation**: Enables precise dynamic and leakage power estimation using actual parasitics.
- **Signal Integrity**: Crucial for detecting crosstalk, IR drop, and noise.
- **Transistor-Level Detail**: Used in analog and custom designs where detail is critical.

---

## 11.3 Structure of a DSPF File

DSPF files contain the following main sections:

- *DESIGN: Design name
- *D_NET: Each net section with resistances and capacitances
- *CONN: Connection points (pins, transistors)
- *CAP: Capacitance values and locations
- *RES: Resistance between nodes
- *END: End of a net

---

Akarshan Arora

## 11.4 Sample DSPF File

```
*BEGIN
*DESIGN MyChip
*TECH tech45nm
*D_NET net1 0.002
*CONN
  X1/A I
  X2/B I
*CAP
  C1 X1/A 1.0e-15
  C2 X2/B 1.2e-15
*RES
  R1 X1/A X2/B 50.0
*END

*D_NET net2 0.0015
*CONN
  X2/C I
  X3/D I
*CAP
  C3 X2/C 1.1e-15
  C4 X3/D 1.3e-15
*RES
  R2 X2/C X3/D 40.0
*END
*END
```

---

## 11.5 How to Read DSPF

Each net begins with *D_NET, followed by:

- **Connections**: Listed in the *CONN section. Each connection includes instance/pin and direction (I/O).
- **Capacitances**: *CAP entries include a name, node, and value in Farads.
- **Resistances**: *RES entries include a name, two nodes, and value in Ohms.

For example:

- R1 X1/A X2/B 50.0 means 50 Ohms resistance between pin A of X1 and pin B of X2.
- C1 X1/A 1.0e-15 means a 1 femtofarad capacitor to ground at node X1/A.

---

Akarsha

## 11.6 DSPF vs SPEF

| Feature | DSPF | SPEF |
|---|---|---|
| Granularity | High (device-level) | Moderate (net-level) |
| Format | Proprietary/Text | IEEE Standard |
| Use Case | Analog, Custom | Digital, Standard-Cell |
| Size | Larger | Smaller |
| Details | Transistor parasitics | Net-based parasitics |

---

## 11.7 Tools That Use DSPF

- **CustomSim**, **Spectre**, **HSPICE**: Use DSPF for SPICE simulations
- **PrimeTime**, **Tempus**: Can use DSPF for delay calculation
- **Power tools**: For accurate switching power analysis

---

## 11.8 Summary

- **DSPF** is a high-fidelity parasitic format
- Used mainly in custom and analog designs
- Captures detailed RC elements for nets
- Enables precise post-layout simulations

Next: We'll dive into **VCD (Value Change Dump)** which handles power and transient analysis.

Akarshan Arora

CHAPTER

TWELVE

# CHAPTER 12: VCD FILES (VALUE CHANGE DUMP)

## 12.1 What is a VCD File?

**VCD (Value Change Dump)** is a standard file format used to log simulation data of digital circuits. It captures how signals change over time during a simulation and is widely supported by simulation tools such as ModelSim, VCS, and NC-Sim.

This format allows engineers to visualize waveforms, debug functionality, and analyze power or toggle activity in tools like GTKWave or Synopsys PrimePower.

---

## 12.2 Why is VCD Important?

- **Functional Debug**: Allows engineers to view internal signal changes.
- **Waveform Viewing**: Inputs, outputs, and internal nets can be visualized over time.
- **Power Analysis**: Used as input to power analysis tools that estimate dynamic power based on toggle activity.
- **Regression Tracking**: Changes in functionality or bugs across simulation runs can be traced.

---

## 12.3 Structure of a VCD File

A VCD file is structured in four main sections:

1. **Header Information**:
   - $date, $version, $timescale indicate metadata about the simulation.
2. **Variable Declarations**:
   - $scope, $var blocks define which modules and signals are logged.
3. **Initial Values**:
   - Specifies the initial values of all dumped variables.
4. **Value Changes Over Time**:
   - Begins with #<time> followed by value changes for variables.

---

Akarshan Arora

## 12.4 Sample VCD File (Complex Example)

```
$date
  April 2025
$end
$version
  Custom RTL Simulation
$end
$timescale 1ns $end
$scope module top $end
$var wire 1 ! clk $end
$var wire 1 " rst $end
$var wire 8 # data_bus[7:0] $end
$var wire 1 $ valid $end
$var wire 1 % ready $end
$upscope $end
$enddefinitions $end

#0
0!
0"
b00000000 #
0$
0%

#5
1!

#10
0!
1"
b00001111 #
1$

#15
1!

#20
0!
b11110000 #
1%

#25
1!

#30
0!
0$
0%
```

## 12.5 How to Read a VCD File

- **Timescale**: The simulation time is measured in 1ns increments.
- **Signal Aliases**: Special symbols like !, ", #, $, % are short identifiers (aliases) for each signal. These are defined in the $var section:
  - ! = clk
  - " = rst
  - # = data_bus[7:0]
  - $ = valid
  - % = ready
- **Binary Values**: For multibit signals like data_bus, the value is given using the b prefix followed by the binary string. Example:
  - b00001111 # → data_bus = 0x0F
  - b11110000 # → data_bus = 0xF0
- **Timestamps**: Begin with #<time>. All value changes listed after a timestamp occur at that simulation time.
- **Reading Signal Changes**:
  - At #0: All signals are initialized.
  - At #10: rst goes high, data_bus becomes 0x0F, and valid becomes high.
  - At #20: data_bus changes to 0xF0 and ready is high.

---

## 12.6 Use in Power Analysis

Power estimation tools like Synopsys PrimePower or ANSYS PowerArtist use toggle information from VCD files:

- **Toggle Rate**: More switching → higher dynamic power.
- **Time-Averaged Activity**: Helps calculate average power.
- **Block-Wise Analysis**: Can be correlated to modules to optimize hotspots.

These tools read signal toggles over time to estimate real dynamic power instead of relying on worst-case assumptions.

---

## 12.7 Tools That Use VCD

- **GTKWave**: Open-source waveform viewer
- **ModelSim/Riviera-PRO**: Generates and reads VCD
- **PrimePower/PowerArtist**: Reads VCD for power analysis
- **Verilator**: Can dump VCD for simulation outputs

---

Akarshan Arora

## 12.8 Summary

- **VCD** is essential for viewing and analyzing signal activity in digital simulations.
- It logs signal values over time using an efficient text-based encoding.
- Enables debugging, visualization, and power analysis.
- Widely supported by simulators and power analysis tools.

Next: We'll explore the **SAIF file format**, which also tracks switching activity but in a more compact form optimized for power tools.

Akarshan Arora

CHAPTER

THIRTEEN

# CHAPTER 13: SAIF FILES (SWITCHING ACTIVITY INTERCHANGE FORMAT)

## 13.1 What is a SAIF File?

**SAIF (Switching Activity Interchange Format)** is a text-based format used to record the **switching activity** of signals in a design, typically after a simulation. It is primarily used for **power estimation** in EDA tools like Synopsys PrimeTime PX or ANSYS PowerArtist.

SAIF captures summarized information such as:

- The number of 0→1 and 1→0 transitions
- Time a signal stays high or low
- Static probabilities

Unlike VCD files which log every signal change, SAIF is **more compact** and simulation-aware. It does not show signal values over time, but rather **statistics** gathered over a simulation duration.

---

## 13.2 Why is SAIF Important?

- **Power Estimation**: Tools use toggle data from SAIF to estimate dynamic power.
- **Faster Analysis**: Smaller than VCD and faster to parse.
- **Supports Hierarchy**: Switching information is stored with hierarchy context, useful for block-level power analysis.
- **Simulation Efficiency**: Reduces overhead on waveform dumping.

---

## 13.3 Structure of a SAIF File

A SAIF file has key sections:

1. **Header**:
   - Simulation time, tool version, and timescale.
2. **Design Hierarchy**:
   - Each module and instance is described with opening and closing (INSTANCE) blocks.
3. **Signal Activity**:

Akarshan Arora

- Each signal is listed with switching statistics:
  - TOGGLE: Number of toggles
  - TC0, TC1: Time signal stayed 0 or 1
  - STATICPROB: Probabilities of 0 and 1

## 13.4 Sample SAIF File (Detailed Example)

```
(SAIFILE
  (SAIFVERSION "2.0")
  (DIRECTION "backward")
  (DESIGN "top")
  (DATE "Thu Apr 10 2025")
  (VENDOR "CustomSim")
  (PROGRAM "vcs")
  (TIMESCALE 1 ns)
  (DURATION 1000)

  (INSTANCE top
    (NET clk
      (TOGGLE 100)
      (TC0 500)
      (TC1 500)
      (STATICPROB 0.5))

    (NET rst
      (TOGGLE 2)
      (TC0 990)
      (TC1 10)
      (STATICPROB 0.01))

    (INSTANCE U1
      (NET in1
        (TOGGLE 10)
        (TC0 800)
        (TC1 200)
        (STATICPROB 0.2))

      (NET out1
        (TOGGLE 12)
        (TC0 700)
        (TC1 300)
        (STATICPROB 0.3))
    )

    (INSTANCE U2
      (NET in2
        (TOGGLE 7)
        (TC0 850)
        (TC1 150)
```

(continues on next page)

(continued from previous page)

```
      (STATICPROB 0.15))

    (NET out2
      (TOGGLE 14)
      (TC0 600)
      (TC1 400)
      (STATICPROB 0.4))
   )
  )
)
```

## 13.5 How to Read a SAIF File

- **INSTANCE** defines hierarchy, like modules and sub-modules.
- **NET** represents each signal.
- TOGGLE: Total number of times the signal changed.
- TC0/TC1: Time the signal remained at 0 or 1 respectively.
- STATICPROB: Statistical probability the signal was 0 or 1 (given as a fraction between 0 and 1).

For example:

```
(NET clk
  (TOGGLE 100)
  (TC0 500)
  (TC1 500)
  (STATICPROB 0.5))
```

This tells us:

- clk toggled 100 times in the simulation.
- It was 0 for 500ns and 1 for 500ns.
- Probability of being high was 50%.

## 13.6 Comparison with VCD

| Feature | VCD | SAIF |
|---|---|---|
| Format | Event-based | Summary-based |
| Size | Large | Small |
| Suitable for | Debug & waveform | Power estimation |
| Readability | Shows exact transitions | Shows overall activity |
| Tools Used | GTKWave, Simulators | PrimeTime PX, PowerArtist |

Akarsh Arora

## 13.7 Summary

- **SAIF files** provide a compact way to capture signal toggle activity.
- Used for power analysis tools to estimate dynamic power.
- Supports hierarchy and efficient data exchange.
- Easy to read and parse.

**Next**: We will explore another useful backend format called **FSDB (Fast Signal Database)**, which is a compact binary waveform format widely used in industry simulations for high-performance analysis and debugging.

Akarshan Arora

CHAPTER

FOURTEEN

# CHAPTER 14: FSDB FILES (FAST SIGNAL DATABASE)

## 14.1 What is an FSDB File?

**FSDB (Fast Signal Database)** is a binary format developed by **Novas (now Synopsys)**. It is widely used in digital simulation and verification environments to store waveform data and signal activity information. Unlike VCD, which is a plain text file, FSDB offers superior **performance**, **storage efficiency**, and **faster access** for large-scale designs.

---

## 14.2 Why is FSDB Important?

- **Efficient Handling of Large Data**: FSDB is optimized for gigabyte to terabyte-scale waveform data.
- **Faster Loading and Navigation**: Tools like Verdi and Debussy use FSDB for high-speed waveform viewing and signal tracing.
- **Hierarchical Data Access**: FSDB maintains full design hierarchy, making it easier to trace signals across modules.
- **Compact Binary Format**: Reduces file size compared to text-based formats.
- **Supports Annotations and Debug Data**: Including assertions, wave traces, and event tracking.

---

## 14.3 Tools that Use FSDB

- **Verdi**: Popular waveform viewer and debugger.
- **DVE (Discovery Visual Environment)**: Integrated with Synopsys tools.
- **VCS, NC-Sim**: Can generate FSDB files with appropriate flags.
- **Joules, PrimePower**: For power analysis based on FSDB toggle data.

---

## 14.4 Generating FSDB Files

In most simulators, FSDB generation is enabled using:

```
$fsdbDumpfile("wave.fsdb");
$fsdbDumpvars;
```

Akarshan Arora

Or using command line flags:

```
vcs testbench.v -debug_all
./simv +fsdbfile+wave.fsdb +fsdbdumpvars
```

This tells the simulator to dump the waveform information into an FSDB file.

---

## 14.5 How to Read FSDB Files

FSDB files are **not human-readable**. You must use visualization tools or converters:

- **Verdi GUI**: Most common tool for visualizing and navigating FSDB data.
- **fsdb2vcd**: Converts FSDB to VCD if text-based access is needed.

```
fsdb2vcd wave.fsdb wave.vcd
```

---

## 14.6 FSDB and Power Estimation

Power analysis tools like **PrimePower** or **Joules** can extract switching activity from FSDB. The tool reads the waveform toggles, applies technology-specific data (e.g., capacitance, voltage), and computes dynamic power.

This makes FSDB a viable alternative to other toggle data formats in power workflows.

---

## 14.7 Summary

- FSDB is a binary waveform format developed for efficient simulation data storage.
- It is widely used in professional EDA flows for debugging, visualization, and power estimation.
- Tools like Verdi rely heavily on FSDB for advanced debugging capabilities.
- While not human-readable, FSDB offers the best performance for large-scale designs.

Next: We will compare **FSDB**, **VCD**, and **SAIF** formats to understand their relative strengths, limitations, and use cases.

Akarshan Arora

CHAPTER

FIFTEEN

# CHAPTER 15: COMPARING FSDB, VCD, AND SAIF FORMATS

## 15.1 Overview

In digital design and verification workflows, waveform and activity formats play a critical role in debugging, visualization, and power analysis. Three of the most widely used formats are:

- **VCD (Value Change Dump)**
- **SAIF (Switching Activity Interchange Format)**
- **FSDB (Fast Signal Database)**

Each of these formats has its own strengths and limitations, making them suitable for different stages and objectives in the VLSI design flow.

---

## 15.2 Feature Comparison Table

| Feature | VCD | SAIF | FSDB |
|---|---|---|---|
| Format Type | Text-based | Text-based (summary) | Binary |
| File Size | Large | Small | Medium to Large (optimized) |
| Readability | Human-readable | Human-readable | Not human-readable |
| Signal Detail | Exact transitions & timing | Toggle counts and stats | Exact transitions & timing |
| Hierarchy Support | Partial | Full | Full |
| Visualization Tools | GTKWave, ModelSim, etc. | Not for visualization | Verdi, DVE |
| Power Estimation | Limited (via toggle count) | Preferred | Supported |
| Speed of Access | Slow | Fast | Very Fast |
| Storage Efficiency | Poor | Excellent | Excellent |

---

## 15.3 When to Use Each Format

- **VCD**:
  - Best for **early debugging** and **functional verification**.
  - Useful when examining signal transitions manually.
  - Inefficient for large designs due to size and parsing time.
- **SAIF**:

Akshan Arora

  - Ideal for **power estimation workflows**.
  - Contains **summarized activity**, not timing-specific.
  - Lightweight, faster to process.
- **FSDB**:
  - Suited for **complex debugging** and **post-silicon analysis**.
  - Works well with advanced EDA tools.
  - Useful in both **waveform tracing** and **power analysis**.

---

## 15.4 Conversion and Interoperability

Sometimes, it's necessary to convert between formats depending on tool support:

- **FSDB to VCD**:

```
fsdb2vcd wave.fsdb wave.vcd
```

- **VCD to SAIF**: Tools like Synopsys VCS can generate SAIF from VCD:

```
vcd2saif -input wave.vcd -output activity.saif
```

---

## 15.5 Summary

Each format has a specific role:

- Use **VCD** for small simulations and signal-level debugging.
- Use **SAIF** for fast and accurate power estimation.
- Use **FSDB** for high-performance simulation data handling and in-depth analysis.

Choosing the right format helps streamline simulation, debugging, and power workflows in VLSI design.

Akarshan Arora